MIE438 Project Proposal:
Autonomous Cup Pong Robot

January 19, 2026

Abanoub Bashara (1010010612)
Solomon Pius (1009975127)
Roy Bou Abboud (1009905462)
Yiyi Xu (1009851402)

1. Problem Overview & Solution Scope

Cup Pong *(otherwise known as Beer Pong)* is a social game consisting of two teams taking turns throwing a ping pong ball into the opposition's cups. Ten cups are placed on both ends of the table, typically in a pyramid formation, with teams standing behind their respective set of cups. Each team, composed of 1-2 players, receives two attempts at sinking a ping pong ball across the table before the turns switch. If a ball is sunk into a cup, that cup is removed from the game, and in order to win, a team must sink a ball into all the opposition's cups. Additionally, if a team sinks both of their shots within an attempt (ie. they didn't miss), they receive an additional two shots as a reward, known as *balls back*, with no limit to how many times this can be done. Hence, if a team receives *balls back* and continually hits their two shots, they can win the game in a single turn, known as a *perfect game*.

The objective of our project is to develop an **Autonomous Cup Pong Robot** capable of playing a *perfect game* without human intervention. An overview of the robot's capabilities includes:

1. Detecting cup geometry and distance from one end of the table
2. Pivoting its launcher in the direction of a desired cup
3. Calculating the required launch speed of the ping pong ball
4. Launching the ball and sinking it into a cup
5. Repeating steps 1-4 until the robot achieves a *perfect game*

Many game mechanics are beyond the scope of the robot yet still crucial to the game's playability. Human interaction to ensure a smooth game is required in the following ways:

- The robot's storage tank must be manually filled with ping pong balls
- Once a cup is eliminated, a human will remove the cup off the table
- After a shot, the robot should standby until it receives the green light from a human to shoot again. This can be phased out in improved designs
- If the robot misses, a human will manually place it in standby after its attempt

1.1 Proposed Project Timeline

Phase 1: Minimum Viable Product (MVP)

A single cup is placed at a fixed, known distance directly in front of the robot. This phase will verify that the core turret and launching mechanisms work. This phase will be considered complete once the system can demonstrate repeatable ball release with controllable velocity and consistent projectile motion.

Phase 2: Controlled Autonomy

Again, a single cup is placed, but now at a random location on the table. The coordinates will be manually provided to the robot. The robot autonomously rotates its base to align with the target and computes the required launch speed to successfully sink the ball. This phase must demonstrate closed-loop control over aiming and velocity.

Phase 3: Vision-Based Target Detection

Again, a single cup is placed at a random location on the table. The robot must now use its camera-based perception to detect the position of the cup and extract the coordinates in order to aim and sink the ball.

Phase 4: Full Game Automation (Perfect Game)
The robot sequentially targets and eliminates all cups on the table, with the only human intervention being cup removal and ball reloading. This phase is complete once the fully autonomous pipeline can consistently win perfect games.

Phase 5: User Experience & Taunting
This phase adds a simple interface to switch between manual cup selection and fully autonomous gameplay. Optionally, audio and/or visual taunting may be implemented.

2. High-Level System Overview

The robot is designed in a modular manner, as demonstrated in the distinct phases outlined in section 1.1. At a high level, the system follows a sense-plan-act architecture: visual information about the environment is processed to determine target locations, a ball trajectory is computed, and the mechanical subsystems execute the shot.

2.1 Overall System Architecture

The system is composed of three interacting subsystems (see figure 1 in the appendix):

1. Perception Subsystem (Phase 3+): identifies cup locations on the table
2. Computation & Control Subsystem (Phase 2+): computes aiming and launch parameters
3. Actuation Subsystem (Phase 1+): physically loads, aims, and launches the ball

This separation allows for the development and testing of subsystems before full system integration.

2.2 Perception Subsystem

A camera is mounted on a stationary base on top of the robot and captures a 2D image of the playing field. This image is processed by computer vision algorithms to detect cup geometry and estimate the coordinates of each cup relative to the robot. These algorithms will either run on an external remote laptop or a Raspberry Pi to decouple from time-sensitive computation (next section). We also suspect that a dual-chip board such as the Arduino Uno Q might be of use in this application.

2.3 Computational & Control Subsystem

Based on the coordinates, this system computes the required turret rotation angle and launch speed using projectile motion models. A low-level language (e.g., C++) running on an ESP-based microcontroller or similar executes real-time control tasks, including motor commands, timing and sequencing.

2.4 Actuation Subsystem

This system consists of a Nerf-style motorized launcher to provide ball velocity, a secondary actuator (servo or solenoid) for ball loading, and a rotating base or turret for horizontal aiming. The motorized launchers spin two flywheels at a set speed to launch the ball. Upon receiving control commands, the robot aligns itself with the target, adjusts launcher speed, feeds a ball, and fires.

3. Hardware (Mechanical / Electrical) Details

3.1 Hardware Overview

The design features a fixed base with a camera attached via a vertical mast; the shooter is mounted on top of the base and rotates the turret independently of the base (See figure 2 in the appendix). Upon activation, a fixed camera takes a picture of the cup field (assuming standard camera placement and

lighting). The compute unit then processes the frame to find cup coordinates and calculates the required turret angle and flywheel RPM to reach the target cup. These metrics are then fed to the embedded controller, which rotates the turret, ramps the flywheel to the command RPM and activates the loader to feed a ball into the spinning wheels for launch.

3.2 Actuation + Mechanical Design

The system utilizes 3 actuators. (1) Two Nerf-style motors that spin flywheels to a given RPM for launch. (2) a turret motor to rotate the shooter heading, (3) a feeder motor driving a pinion mechanism to push one ball at a time into the spinning flywheel. The flywheel RPM will be controlled by varying motor drive power, while the turret and feeder move at a slow, consistent rate to avoid structural stress. The vertical shooting angle of the turret will be fixed at 50-60 degrees, striking a good balance between maximizing shot distance and ball height as well as minimizing drag.

3.3 Sensors + I/O Requirements

The system's primary sensor is a mounted camera that captures an image on user command. That image is then processed on a Raspberry Pi or remote Laptop and outputs setpoints to the MCU over USB serial (Target turret angle & Flywheel RPM). Table 1 in the appendix shows estimates on the quantitative I/O analysis.

3.4 Compute + Embedded Platform

The system uses a Raspberry Pi 4 Model B or Remote PC for camera capture, vision, trajectory/RPM/angle calculation and an ESP32 for real-time actuation (turret servo control, flywheel RPM command generation, feeder sequencing and standby logic). Since the ESP32 is 3.3V logic, if any hardware requires 5V signalling, level shifting may be required. Power will come from a wall adapter with separate regulated rails for logic, 5V for Pi, 3.3V for ESP32, and motors as required.

3.5 Power + Key Components/BOM

The system will be powered via a wall adapter that steps down to the required rails for the ESP32, servos, and flywheel motors. Safety hardware will include a power switch plus fuse, and a fuse holder in line with the supply. Current BOM can be found in the appendix.

4. Software / Controls / UI Details

The software required for this project goes beyond the scope of this course, but we believe proper implementation is needed to manifest our vision of what a fully autonomous cup pong robot should achieve.

4.1 Vision

As mentioned before, the software is responsible for producing control commands for where the shooter needs to aim. We first need to pre-process the image by removing the tilt effect from the camera, and then extracting the location of the cups. To achieve this, there are 2 methods we can explore. The first method is using existing libraries for computer vision, such as OpenCV/cv2 in Python, and using least squares regression to fit circles on the rims of the cups. The center of the circles should be the coordinate of the cup and thus the shooter should aim there. A technique that can make this approach more effective is to apply some sort of retroreflective paint to the rim of the cups, and using a camera that can pick up said retroreflective paint. An alternative method, which might yield better results, is to manually train our own vision model on labelled data of images of shots. This

will be more tedious to do but might yield better results if we are unable to fit OpenCV in our use case.

4.2 Controls

Once the coordinates of the cups are determined, we can select 1 cup to shoot at first. There are again 2 ways to convert coordinates to turret angle and shooter RPM. We can either solve for them analytically using inverse kinematics, which in theory should work 100% of the time. However, under real conditions with non-negligible drag and spin on the ping pong balls, idealized assumptions might not hold up. A different method is to manually find pivot angle + shooter RPM combos by testing different shots, and then interpolating those points for all the cup locations.

The turret motor should have an encoder, so we're able to use a closed loop control such as PID to get it to its desired location. For the flywheel motors, we can use an RPM sensor to achieve closed loop speed control, with the control input being the PWM duty cycle. For any known target distance, the system determines a desired motor RPM, and the PWM signals are adjusted using the RPM sensor to achieve this desired value before the ball is launched.

4.3 UI

There needs to be some way for us to interact with the robot once it starts playing the game. Most likely we will have a simple push button to allow the robot to start shooting once it's been reloaded and between shots.

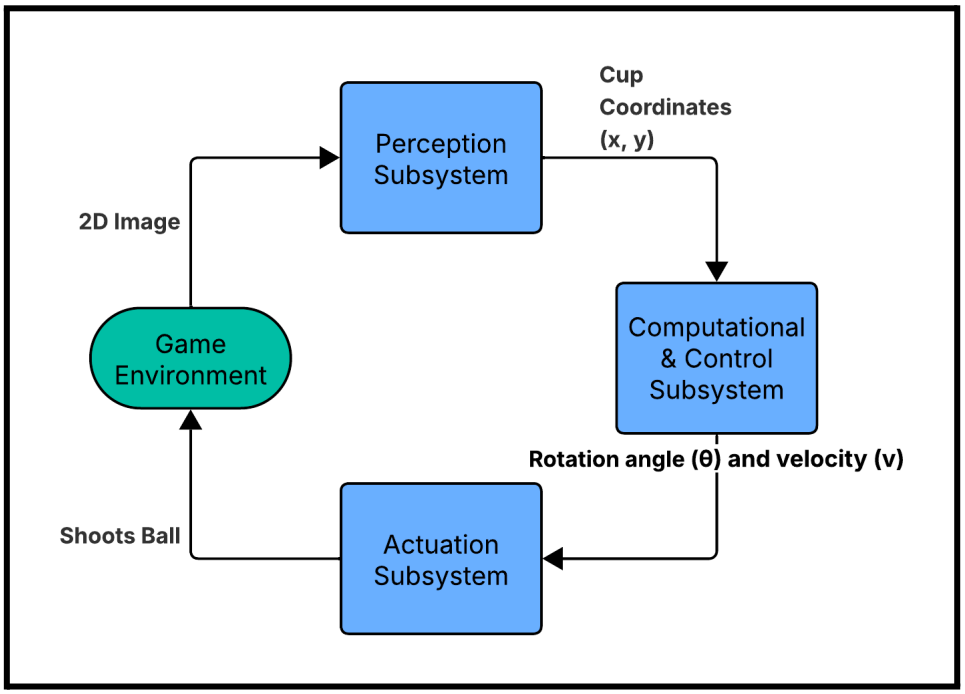
If the robot is able to consistently win games, we can add an LCD display to personify the robot and add a competitive flair. It can taunt the enemy when it makes shots and emote as its shooting. Alternatively it can be used as a status light, showing us what state the robot is in (spinning up shooter, aiming, processing, etc).

5. Conclusion

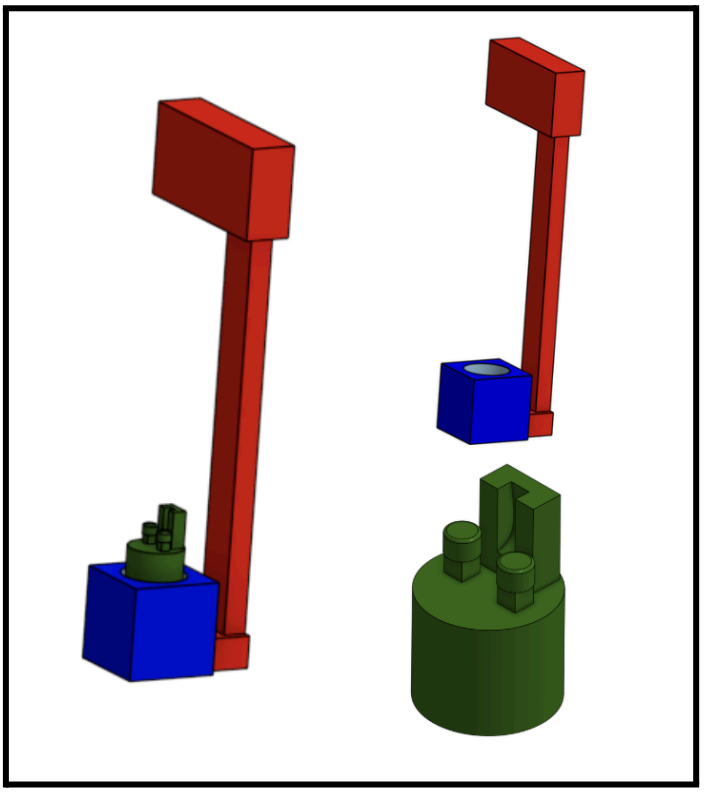
Overall, we believe this embedded system design is feasible under the scope of this course and we are highly interested in pursuing this project. We consider achieving Phase 2 (Controlled Autonomy) a complete embedded systems project, with subsequent phases adding additional functionalities that greatly increase the system's performance. It is important to note that 3D printed structural and mechanical components such as the base, gears, etc are not part of the embedded systems design work we are planning to pursue, and hence, we will be using pre-existing models found on the internet. Additionally, we plan on always testing and demonstrating our system in stable environments with good lighting, mitigating unintended factors (e.g. occlusions) that may decrease the robot's performance. Finally, in terms of teaching team feedback, what would help us the most is insight on whether using a Raspberry Pi, Arduino Uno Q, or remote PC as a replacement or in combination with an ESP32 DevBoard are admissible options for the course and this project.

6. Appendix

A. **External Link 1: MIE438 Project - Bill of Materials**



B. **Figure 1: Flow Chart Depicting the Different Subsystems & How They Interact**



C. **Figure 2: Simple CAD design to show proof of concept. (Blue: base of system. Red: Camera stand. Green: Individually rotating turret)**

Signal	Type	Where	Estimated range/requirement
Camera frame	USB/CSI video	Pi	1280×720 (or 640×480) single-frame capture; needs consistent lighting for detection
User ‘shoot’ trigger	Digital GPIO	Pi	Button input (0/1), debounced (~20–50 ms)
Turret angle (tracked)	Software state	MCU	0–180° (or ±90°), updated from last command; resolution target ~1–2°
Setpoints (angle, RPM, fire)	USB serial	Pi → MCU	~5–10 Hz command updates; target end-to-end latency < ~250 ms
Flywheel command	PWM(to motor driver)	MCU → driver	RPM setpoint ~0–20,000 RPM (estimate), resolution ~100–250 RPM
Turret motor command	Step/Dir or PWM	MCU → driver	Slow slew target ~10–30 °/s (estimate)
Feeder command	PWM + dir (or timed on/off	MCU → driver	One-ball feed cycle ~0.5–1.5 s (estimate), return-to-zero after shot

D. Table 1: Hardware I/O Map